Experiment 9(b) - Parallel Processing Using OpenMP

Step-by-Step Practice Guide for Ubuntu

Aim: To implement parallel processing using OpenMP in Linux.

What you will do: install the OpenMP development library, compile C programs with **-fopenmp**, and run parallel reductions for sum, maximum, product and average.

Important source note

The record's Step 2 heading says **Install python 3**, but the screenshot under that step actually runs **sudo apt-get install libomp-dev**. This guide preserves the command shown in the record because OpenMP is the library used by the programs.

Step 1 - Update Ubuntu

```
sudo apt update
```

Step 2 - Install the OpenMP library

```
sudo apt-get install libomp-dev
```

You also need GCC. Check it with:

```
gcc --version
```

Step 3 - Program 1: Total sum and maximum element

Create the file:

```
nano sumandmax.c
```

The source screenshot in the record contains two consecutive OpenMP pragmas. That form can fail with current compilers. The runnable practice version below keeps the same intended operations but combines both reductions into one directive.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int arr[] = {1,2,3,4,5};
    int sum = 0;
    int max = arr[0];

    #pragma omp parallel for reduction(+:sum) reduction(max:max)
    for (int i = 0; i < 5; i++) {
        sum += arr[i];
        if (max < arr[i])
            max = arr[i];
    }

    printf("max %d ", max);
    printf("\nsum %d\n", sum);
    return 0;
}
```

Save the file, then compile:

```
gcc -fopenmp -O2 sumandmax.c -o sumandmax.out
```

Run with four OpenMP threads, as shown in the record:

```
OMP_NUM_THREADS=4 ./sumandmax.out
```

Expected output:

```
max 5
sum 15
```

Step 4 - Program 2: Product and average

```
nano prodandavg.c
```

Runnable practice version matching the intent and output in your record:

```
#include <stdio.h>
#include <omp.h>

int main() {
    int arr[] = {1,2,3,4,5};
    int pro = 1;
    int sum = 0;

    #pragma omp parallel for reduction(*:pro) reduction(+:sum)
    for (int i = 0; i < 5; i++) {
        pro *= arr[i];
        sum += arr[i];
    }

    printf("product %d ", pro);
    printf("\navg %d\n", sum / 5);
    return 0;
}
```

Compile:

```
gcc -fopenmp -O2 prodandavg.c -o prodandavg.out
```

Run:

```
OMP_NUM_THREADS=4 ./prodandavg.out
```

Expected output:

```
product 120
avg 3
```

What does reduction do?

• **reduction(+:sum)** gives each thread a private partial sum and safely combines the partial sums at the end.

• **reduction(*:pro)** safely combines partial products.

• **reduction(max:max)** combines the largest value seen by the threads.

• **OMP_NUM_THREADS=4** asks OpenMP to use four threads for the parallel region.

Quick exam rehearsal

```
sudo apt update
sudo apt-get install libomp-dev

nano sumandmax.c
gcc -fopenmp -O2 sumandmax.c -o sumandmax.out
OMP_NUM_THREADS=4 ./sumandmax.out

nano prodandavg.c
gcc -fopenmp -O2 prodandavg.c -o prodandavg.out
OMP_NUM_THREADS=4 ./prodandavg.out
```

Common mistakes

• Forgetting **#include <omp.h>**.

• Compiling without the **-fopenmp** option.

• Writing the reduction operator incorrectly.

• Using integer division for the average and expecting a decimal result. The record's array gives exactly 3, so integer output is fine here.

Result

Thus the implementation of parallel processing using OpenMP in Linux is completed successfully.